

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Программирование графических процессоров»

Обработка изображений на GPU. Фильтры.

Выполнил: Орусский В.Р.
Группа: М8О-406Б-21
Преподаватели: А.Ю. Морозов

Москва, 2025

Условие

Цель работы. Научиться использовать GPU для обработки изображений.

Использование текстурной памяти и двухмерной сетки потоков.

Вариант 4. SSAA (Super Sampling Anti-Aliasing).

Необходимо реализовать избыточную выборку сглаживания. Исходное изображение представляет собой “экранный буфер”, на выходе должно быть сглаженное изображение, полученное уменьшением исходного.

Входные данные. На первой строке задается путь к исходному изображению, на второй, путь к конечному изображению. На следующей строке, два числа w и h -- размеры нового изображения, гарантируется, что размеры исходного изображения соответственно кратны им ($w * h \leq 4 * 10^8$).

Программное и аппаратное обеспечение

GPU:

- Название NVIDIA GeForce GTX 1660 TI
- Compute capability: 7.5
- Графическая память: 2147155968
- Разделяемая память: 49152
- Константная память: 65536
- Количество регистров на блок: 32
- Максимальное количество нитей: (1024, 1024, 64)
- Максимальное количество блоков: (2147483647, 65535, 65535)
- Количество мультипроцессоров: 24

Сведения о системе:

- Процессор: Intel(R) Core(TM) i5-10600KF CPU @ 4.5000MHz
- ОЗУ: 16 ГБ
- SSD M2 1 ТБ

Программное обеспечение:

- OS: Windows 10 + WSL (Ubuntu 20.04)
- IDE: Visual Studio Code
- Компилятор: nvcc

Метод решения

В данной работе используется Метод суперсэмплирования SSAA (Supersample Anti-Aliasing). В идее данного алгоритма содержится первичное увеличение изображения, вычисление усреднённых значений пикселей, а потом сжатие до желаемого размера. Данный метод активно применяется в игровой индустрии, используется для уменьшения артефактов, «лесенок».

Описание программы

- Считывает путь к входному и выходному файлам, а также целевые размеры изображения ($\text{resWidth} \times \text{resHeight}$) из консоли;
- Загружает из бинарного файла «*.data» размеры исходного изображения ($\text{width} \times \text{height}$) и массив пикселей `rgba_data` в формате RGBA;
- Вычисляет коэффициенты сжатия (`widthRatio`, `heightRatio`) для перехода от исходного разрешения к целевому;
- Выделяет память на GPU, создает массив CUDA (`cudaArray`) и копирует в него входные данные;
- Настраивает текстурный объект (`cudaTextureObject_t`) с режимом чтения пикселей без нормализации координат и с использованием крайних значений;
- Запускает ядро `kernel` на сетке из 16×16 блоков и 16×16 потоков, где для каждого выходного пикселя усредняются цвета из области исходной текстуры;
- Копирует результат с GPU обратно на хост и записывает его в выходной бинарный файл;
- Освобождает выделенную память и завершает работу.

Примеры работы

Исходное изображение:



Обработанное изображение:



Для сравнения использовался скриншот из видеоигры, где как раз зачастую применяются методы, подобные SSAA.

Результаты

Размер изображен ия	CPU	<<<(16,16),(16,16)>>>	<<<(32,32),(32,32)>>>	<<<(64,64),(32,32)>>>	<<<(128,128),(32,32)>>>	<<<(512,512),(32,32)>>>
64×64	22.5	0.045	0.058	0.1	1.1	3.4
256×256	230	0.06	0.12	0.5	1.2	3.8
1024×1024	4200	0.11	0.25	0.77	1.37	3.96
2048×2048	25000	0.28	0.5	0.86	1.6	4.29
10000×10000	560000	3.2	3.0	2.7	4.1	7.7

Выводы

В рамках работы на практике узнал, какие алгоритмы применяются для обработки изображений в различных компьютерных программах, с которыми сталкивался лично. Была изучена текстурная память. Программа эффективно использует текстурную память CUDA через объект `cudaTextureObject_t` для ускорения обработки изображения в методе SSAA, обеспечивая быстрый доступ к данным пикселей при усреднении значений в ядре `kernel`. Оптимизация достигается за счет использования текстурной памяти, которая минимизирует задержки при многократном чтении данных исходной текстуры, особенно при избыточной выборке для сглаживания, как в случае с областью $\text{widthRatio} \times \text{heightRatio}$ для каждого выходного пикселя.